新能源车队调度仿真：建模思路、规模参数与得分含义

党涵

1 总体思路

题目要求用**图结构**表示道路并完成**路径规划**，在动态到达的任务下进行车队调度。本实现采用以下抽象层次。

1.1 道路网络（图模型）

- 将城市抽象为 $r \times c$ 的**网格图**：每个交叉点为一个顶点，四邻连接（上下左右）。
- 边权为相邻顶点间的**欧氏距离**（网格轴相邻边权为 1）。
- 顶点按行主序编号，0 号节点为**中央仓库（车场）**。
- 使用 **Dijkstra** 单源最短路；从某顶点出发的距离行按需计算并**缓存**，路径由父指针回溯得到（大图避免 Floyd 的 $O(|V|^3)$）。

1.2 时间推进与任务生成

- 仿真时间为离散时间步，步长 $\Delta t = 0.5$。
- 每个时间步以概率 $p_{\text{spawn}} \cdot \Delta t$ 独立生成一个新任务。
- 每个任务包含：产生时刻、目标顶点、货物重量（区间内均匀随机）、截止时间（产生时刻加上区间内均匀随机的裕量）。

1.3 车队与能耗

- 每辆车有**电量上限** $B_{\max}$、**载重上限** $L_{\max}$；每完成一票配送，车上对应载重释放。
- 行驶距离 d 的能耗近似为 $E(d) = \rho \cdot d$，其中 ρ 为**单位距离能耗**。
- 车速用于将距离换算为行驶时间：$t_{\text{travel}} = d/v$。

1.4 基线调度策略（`fleet_simulation.py`）：仓库多票装车 + 最近邻配送顺序

车辆**仅在仓库**且空闲时派发新的一批任务（代码中 `node` 为仓库且无在送批次）。模型假定**所有货物在仓库一次装车**，再沿路连续送票，**本批全部送完后**再执行回仓（与“每送一票必须先回仓”不同）。具体为：

1. **装车（贪心）**：在待分配、已出现且未过期的任务中，按**重量从大到小**依次装入，直至再加一票会超过 $L_{\max}$，得到本趟**批次**（可多票）。同重量时排序顺序由实现细节决定（稳定排序下保持候选表原有次序）。

2. **配送顺序（最近邻）**：以仓库为起点，在批次内反复选择**距当前位置图最短路最短**的下一配送点，直至排完访问序（启发式，非最优 TSP）。

3. **电量可行性**：计算**仓库** → **各配送点（按上述顺序）** → **回仓库**的总路程能耗；若超过当前电量，则反复从批次中**去掉当前最轻的一票**并重新最近邻排序，直至总能耗不超过电量，或批次退化为**仅一票**（仍不足则允许经充电站补能，逻辑见下节）。

4. **在途**：按访问序分段行驶；每一段均为“当前位置 → 下一配送点”，段内仍可采用途经充电站路径。每送完一票仅卸下该票重量；若尚有余票，则**从当前配送点直接**驶向下一点，无需回仓。

代码中另保留单票选取函数 `pick_task_max_weight`，便于与“只装一票”等策略做对比，主流程为 `pick_batch_greedy_max_weight`。

1.5 元启发式对比实现（`fleet_metaheuristic.py`）

在**与基线相同的** `SimConfig`、任务生成随机种子及 `self._rng` 消费顺序下，通过子类 `MetaHeuristicFleetSimul` 仅替换**装批平局规则**与**批次内访问序**；仿真主循环、充电与路径逻辑仍继承基类。元启发式内部随机数使用独立种子（`seed+` 常数偏移），避免干扰任务流。

装车（批次层面） 仍按**重量从大到小**再在载重内贪心装入，与基线结构一致；区别为**同重量**任务按**截止时间先后**（EDD），再按任务编号 `tid` 打破平局，对应 `pick_batch_weight_then_edd`。这样在不推翻“重货优先装满车”的前提下引入紧迫度信息。

批次内路径优化（元启发式核心） 派发批次时记录当前仿真时刻 t_0，对批次内访问序 π 定义**与总分系数对齐的代理代价**（用于优化，非仿真直接加分公式）：

$$J(\pi) = \alpha_d\, D(\pi) + \alpha_{\text{late}}\, L(\pi). \tag{1}$$

其中 $D(\pi)$ 为**仓库** → **按** π **依次送达各点** → **回仓库**的图最短路总里程（与电量检验所用回路长度一致）；$L(\pi)$ 为按 π 顺序、以 t_0 为起点用行驶时间累加到达各点时，**正迟到时间**之和，即对每个任务取 $\max\{0, t_{\text{arr}} - t_{\text{dl}}\}$ 再求和（t_{arr} 由连续最短路距离除以车速 v 递推）。

算法流程

- 若批内任务数 $n \le 6$，对全排列枚举，取 $J(\pi)$ 最小的访问序（小规模精确最优）。
- 若 $n > 6$，采用**模拟退火**：初解在**基类最近邻序**与**按截止时间排序的全序列（EDD）**中取 J 较小者；邻域包含**交换两位置**、**反转一段（2-opt 型）**、**单点移位（Or-opt-1）**；对交换移动辅以**短禁忌表**，长期无接受则**升温重启**降温轨迹。
- 电量不可行时的**丢票重排**逻辑与基线相同：仍按最轻票剔除后重新优化访问序（调用同一套批次内优化子程序）。

对比实验脚本 `fleet_metaheuristic.py` 中 `run_controlled_comparison()` 逐场景先跑基线 `FleetSimulator` 再跑子类，并校验任务流签名（产生时刻、目标节点、重量序列）一致，最后输出两套 `summarize` 得分差，便于报告中的控制变量说明。

1.6 充电与排队

- 对**每一段**行驶（含首段出仓、批次内相邻配送点之间、以及送完本批后的回仓段），若沿最短路直达的能耗超过当前电量，则先到**图距离最近**的充电站补能后再继续该段。
- 每个充电站有有限个**并发槽位**（代码中为 2）。充电用时间区间 $[t_{\text{start}}, t_{\text{end}})$ 预约；槽位已满时推进到**最早能开始充电的时刻**，体现**排队与负荷**。

2 各规模场景的具体数量

四组场景除下表所列参数外，还共享：

$$B_{\max} = 500, \quad L_{\max} = 250, \quad \rho = 0.8, \quad v = 1.0, \quad P_{\text{charge}} = 75.0,$$

$$\alpha_{\text{early}} = 10.0, \quad \alpha_{\text{late}} = 15.0, \quad \alpha_d = 0.01.$$

充电站每站点并发槽位数为 2。

表 1: 四种仿真规模的显式参数（与 `preset_scenarios()` 一致）

名称	r	c	$\lvert V\rvert = rc$	车辆数	充电站数	T_{sim}	p_{spawn}	重量区间	截止裕量区间	`seed`
SMALL	25	25	625	10	10	1000	0.40	$[5, 40]$	$[125, 275]$	1
MEDIUM	50	50	2500	20	20	1500	0.60	$[5, 75]$	$[100, 300]$	2
LARGE	80	80	6400	40	40	2000	0.75	$[10, 125]$	$[90, 250]$	3
XL_STRESS	100	100	10000	30	25	1750	1.10	$[15, 150]$	$[60, 175]$	4

说明： $|V|$ 为顶点数；边数约为 $2rc - r - c$。**XL_STRESS** 为第四档场景名称：**XL** 表示网格规模最大（100×100）；**STRESS** 表示相对同规模地图**加压**——任务到达率 p_{spawn} 更高、截止裕量区间更紧、车辆与充电站数量相对偏少，用于观察调度在拥堵、易超时情形下的表现，与前三档“常规规模”形成对照。

3 最终得分的含义与计算公式

最终得分 S 为**标量评价指标**，对应大作业中“完成越早、路径越短越高分，超时扣分”的定性要求。实现为**可加**形式。

3.1 已完成任务的贡献

设任务在 t_{fin} 完成，截止 t_{dl}，重量 w，本任务相关行驶距离为 d_{task}。多票同车时，d_{task} 取**送达该票的这一小段路**的最短路长度（上一停靠点 $\to$ 该任务顶点，若经充电站则含充电前路段的计程方式与代码 `travel_distance` 一致），而非整趟多点的总里程。

$$\Delta S_{\text{done}} = \begin{cases} \alpha_{\text{early}}\, w - \alpha_d\, d_{\text{task}}, & t_{\text{fin}} \le t_{\text{dl}}, \\ -\alpha_{\text{late}}\,(t_{\text{fin}} - t_{\text{dl}}) - \alpha_d\, d_{\text{task}}, & t_{\text{fin}} > t_{\text{dl}}. \end{cases} \tag{2}$$

3.2 未完成（过期）任务的惩罚

待分配任务在仿真中首次被发现已超过截止时间时记一次惩罚：

$$\Delta S_{\text{expired}} = -\alpha_{\text{late}}\,(t - t_{\text{dl}}). \tag{3}$$

3.3 如何解读 S

- S **无绝对物理单位**，是人为设定的效用代理，用于同一套参数下比较策略或随机种子。
- **越大越好**；大量过期或迟到时 S 可显著为负（罚分主导）。
- 若修改 $\alpha_{\text{early}}, \alpha_{\text{late}}, \alpha_d$，数值尺度会变，比较时需对齐系数。

4 静态 MILP（fleet_gurobi_static.py）与离散仿真（fleet_simulation.py）的对照

除**提前已知**全部任务的出现时刻、目标顶点、重量与截止时间（上帝视角）外，仍希望尽可能共用同一“世界”。本节说明**哪些条件与仿真一致**、**哪些必然不一致**，避免将 Gurobi 输出与 `run()` 轨迹做**逐事件等价**的误读。实现以 `gurobipy` 求解混合整数规划；具体变量与约束以代码为准。

4.1 数据与随机世界：可与仿真对齐的部分

- **场景参数**：与 `preset_scenarios()` 中 `SimConfig` 一致（网格规模、车辆数、充电站数、T_{sim}、p_{spawn}、重量与截止裕量区间、$B_{\max}, L_{\max}, \rho, v, P_{\text{charge}}$ 及得分系数等）。
- **任务列表与充电站坐标**：代码中通过构造 `FleetSimulator(cfg)` 后**仅执行与 `run()` 相同的任务生成循环**（时间步长 $\Delta t = 0.5$，每步以概率 $p_{\text{spawn}} \cdot \Delta t$ 生成一单），**不执行**车辆派发与 `step()` 中其余逻辑。由此得到的任务序列与 `FleetSimulator.__init__` 中已确定的**仓库顶点**、各充电站**图节点编号**（`ChargingStation.node`）与完整仿真在“仅生成任务”意义下一致，从而保证**最短路距离矩阵**所依赖的端点与仿真相同。
- **图与最短路**：与仿真一样在网格邻接图上用 Dijkstra 得到任意两点间最短路长度；行驶时间仍取 d/v。
- **能耗与补电时长**：弧上耗电仍用 $\rho \cdot d$；在充电站将电池从到达电量充至 $B_{\max}$ 所需时间，与仿真中按功率 P_{charge} 由能量缺口换算的思路一致（代码中为线性约束 $\texttt{dur} \cdot P_{\text{charge}} \geq B_{\max} - B_{\text{arr}}$ 等）。
- **载重**：静态模型中每辆车**单趟路径**上所承载任务重量之和 $\leq L_{\max}$，对应仿真里“在仓库一次装齐本批再出发、本批总重不超过载重上限”的设定。
- **并发槽位数**：每个物理充电站在任意时刻“同时在充”的车辆数上限为 2，与仿真里 `slots=2` 的**数量**一致（实现机制不同，见下节）。

4.2 模型结构：与仿真不一致或仅为近似的部分

以下差异来自**静态优化**与**离散事件仿真**的不同范式，一般不能仅靠调参消除。

（1）动态多趟 vs. 静态单趟 仿真中车辆在仿真期内可多次回仓、再装新批、再出发，形成**多条**“出仓—送批—回仓”轨迹；当前 MILP 为**每辆车一条**从起点仓到终点仓的闭合路径抽象（含可选充电节点），相当于对**一整段连续作业**做一次性规划，**不重现**“回仓后再派下一批”的多轮决策过程。因此即便任务全集相同，**时间轴上的决策次数与仿真也不相同**。

（2）充电排队：连续时间预约 vs. 离散时间槽容量 仿真中充电通过时间区间 $[t_{\text{start}}, t_{\text{end}})$ 预约，在槽位占满时按**最早可开始时刻**推进，是会话列表驱动的**连续时间**排队。MILP 中将时间轴按步长 Δ_q（代码参数 `queue-delta`）分段，用二进制变量刻画充电区间与每个时间槽是否相交，并对每个物理站、每个槽施加**同时在槽内占用数** ≤ 2。这是对“两槽并行”的**离散近似：不能保证**与仿真中每一次 `_reserve_charge`、`_next_charge_start` 得到的开始/结束时刻逐次相同；Δ_q 越小通常越细、变量越多，仍未必与连续预约**事件级等价**。

（3）路径段与失败回退 仿真中 `_begin_leg_from_to` 在电量、充电站可达性等条件下可能失败并触发**批次回滚**等逻辑；MILP 通常假设所选弧在能量与时间上可行，**不包含**与仿真完全同构的回滚分支。

（4）目标函数与仿真得分 S 仿真得分 S 按第 3 节在任务完成、过期等**离散事件**上累加；MILP 目标为**线性代理**（如服务任务的重量的按时奖励系数之和、减去迟到项、距离惩罚及未服务惩罚等），用于在静态假设下求较优路径，**数值上不等于**同一次仿真运行结束后的 S，也不宜在无控制实验设计下直接宣称“同分”。

（5）任务子集与规模 为控制整数规划规模，实现中常只对按重量排序后的**前若干条**任务建变量（参数 `max-tasks`）；仿真则对**全部**到达任务执行在线规则。对比报告时需说明 MILP 求解的是**子问题**还是经筛选后的实例。

4.3 对照小结表（便于报告引用）

表 2: 静态 MILP 与 `fleet_simulation.py` 主要对照（除上帝视角外）

项目	仿真（动态）	静态 MILP（Gurobi）
任务与充电站几何	可与 `FleetSimulator` 仅生成任务时对齐（同 `seed`、同配置）。	
最短路、v、ρ、$B_{\max}$、$L_{\max}$、P_{charge}	定义一致，用于距离、时间与能耗。	
决策时刻	每步 `step` 在线派车、装批、路径段	一次性离线优化（已知全集）。
车辆路径条数	多趟回仓、多批任务	当前实现为每车单条闭合路径（单趟抽象）。
充电排队	连续区间 + 槽位预约 + 推进最早开始时刻	时间槽上的容量约束（≤ 2），离散近似。
得分	事件累加 S（第 3 节）	代理目标，与 S 不完全同一函数。

4.4 如何撰写对比结论

建议在报告中分三点写清边界：**(i)** 数据层（任务、仓库、充电站节点、路网与参数）与仿真可对齐；**(ii)** 优化层为静态、单趟抽象 + 离散排队近似，与 `run()` 的**多轮动态过程**及**连续充电预约**不完全同构；**(iii)** 数值对比应以“同实例下的上界/参考解/策略差异说明”为主，避免将 MILP 目标值与仿真 S 直接等同为同一标尺。

5 真实路网扩展（OpenStreetMap）与“地图”可视化原理

除网格抽象外，代码中还提供将**同一套车队模型**（多车、电量、充电排队、动态任务与得分，见前文）迁移到 **OpenStreetMap（OSM）道路几何**上的实现：`fleet_osm.py`、`osm_graph.py`，以及仅负责拉数导出的 `osm_fetch_demo.py`。本节说明**使用了哪些库/接口**，以及**从数据到屏幕**的原理，避免与“嵌入百度/高德瓦片地图”一类方案混淆。

5.1 使用了什么：无专用地图渲染 SDK

- **数据来源与协议：OpenStreetMap** 社区数据；通过公共 **Overpass API** 以 `POST` 方式查询。请求与解析使用 Python **标准库** `urllib`、`json`，**不依赖** `folium`、`leaflet`、`osmnx` 等 GIS/前端地图库。
- **图构建与最短路:**在自建的无向加权图上调用与主仿真相同的 **Dijkstra**（实现于 `fleet_simulation.py` 的邻接表版本），同样采用**按源点缓存距离行**。
- **动态可视化：**使用 Python 标准库 **`tkinter`** 的 `Canvas` 逐帧绘制折线段与图标。**没有**加载卫星/街道**瓦片底图**；屏幕上的“地图”是**矢量路网** + **业务图层**在画布上的几何投影。

5.2 从 Overpass 响应到图 $G = (V, E)$

1. 在经纬度边界框（bbox：南、西、北、东）内查询带 `highway=*` 的 `way`，并使用 `out geom`；取得每条道路中心线的**折线顶点序列**（经纬度）。
2. 将相邻几何点 $(\lambda_1, \varphi_1) \to (\lambda_2, \varphi_2)$ 连成**路段**；对端点坐标做**固定小数位量化**（如保留到 10^{-6} 度）以合并数值噪声，得到抽象顶点集 V。
3. 每条无向边连接两个量化后的端点，边权取两点间**大圆距离的球面近似（Haversine）**，记地球半径为 R（代码中取 $R \approx 6371000\,\mathrm{m}$），则
$$a = \sin^2\frac{\Delta\varphi}{2} + \cos\varphi_1\cos\varphi_2\sin^2\frac{\Delta\lambda}{2}, \qquad d_{uv} = 2R\arcsin(\min\{1, \sqrt{a}\}), \tag{4}$$
其中 $\Delta\varphi, \Delta\lambda$ 为弧度差。由此**边权单位为米**，与网格版“抽象距离 1”不同；`SimConfig` 中车速 v、单位距离耗电 ρ 在 OSM 模式下解释为 **m/s** 与 **每米耗电**，时间轴仍为秒，以保持与 `step()` 离散推进同一套接口。
4. 取道路网络的**最大连通分量**，并在该分量上选取**仓库**顶点（实现上偏向靠近点集几何中心且度数较高，以减少“死胡同仓”）；任务仅出现在从仓库**最短路距离有限**的顶点上，保证派发有意义。

5.3 继承 `FleetSimulator` 的含义

`FleetSimulatorRoad` 以子类方式**替换图的来源与仓库/候选点初始化**，**不重写** `step()`、分段行驶、充电预约与得分累加等核心逻辑。因此 OSM 模式与网格模式在**控制与事件语义**上一致，差异主要体现在**拓扑与度量**由规则网格变为**稀疏平面路图**。

5.4 动态画面如何把经纬度画到 Tk 画布上

记当前子图所有顶点经纬度落在经度区间 $[\lambda_{\min}, \lambda_{\max}]$、纬度区间 $[\varphi_{\min}, \varphi_{\max}]$，画布上预留绘图区宽 W_p、高 H_p（像素），左下角映射原点像素偏移为 (o_x, o_y)。则对任意顶点 (λ, φ)，采用**线性仿射 + 纵轴翻转**（屏幕 y 向下为正）：

$$x = o_x + \frac{\lambda - \lambda_{\min}}{\lambda_{\max} - \lambda_{\min} + \varepsilon} W_p, \qquad y = o_y + \frac{\varphi_{\max} - \varphi}{\varphi_{\max} - \varphi_{\min} + \varepsilon} H_p. \tag{5}$$

ε 为防止除零的小常数。路网层：对每条边 $(u, v) \in E$ 连接其两端点在画布上的直线段即可。车辆位置：仿真仍输出最短路顶点序列及每段时间区间；可视化侧在相邻顶点间按**已行驶弧长占该弧总长**的比例做线性插值（与 `fleet_visual.py` 中沿路径插值思想相同，只是把“格子中心坐标”换成上式投影坐标）。

5.5 小结（与主文网格模型的关系）

网格版回答“抽象图上的调度原理”；OSM 扩展回答“若把**真实道路几何**当作图、把**米**当作距离单位，同一调度模型如何运作以及如何**自绘**动态演示”。二者**不共用**同一套 `rows×cols` 可视化，但**共用**同一套车队状态机接口。

6 小结

网格图 + Dijkstra（行缓存）、离散时间动态任务、**仓库多票装车（重量贪心）与最近邻配送序、送完一批再回仓**、分段路径下的充电站槽位与区间预约，以及按时奖励、迟到/过期惩罚、按段路程惩罚构成总分 S。上述基线行为以 `fleet_simulation.py` 为准。**元启发式对比**在 `fleet_metaheuristic.py` 中实现：装车为同重 EDD 平局；批内序为小规模全排列最优或模拟退火最小化 $J = \alpha_d D + \alpha_{\text{late}} L$，与得分系数对齐以便改进访问序时兼顾路程与紧迫度。**静态 MILP** 在 `fleet_gurobi_static.py` 中实现：与仿真可对齐任务与充电站布局及主要物理参数，但在多趟动态过程、充电排队机制与目标函数上与仿真**不等价**，详见第 4 节。**真实路网扩展**见第 5 节：OSM + Overpass（标准库 HTTP）构图，`tkinter.Canvas` 经纬度线性投影动态演示（无瓦片地图 SDK）。

7 电量消耗的具体参数与公式（仅追加说明）

本节将代码中的电量逻辑写成显式公式，便于在报告中直接引用。参数与实现对应 `fleet_simulation.py`。

7.1 参数定义（默认场景共用）

$$B_{\max} = 500, \quad \rho = 0.8, \quad v = 1.0, \quad P_{\text{charge}} = 75.0.$$

其中：

- $B_{\max}$：电池容量上限（满电值）；
- ρ：单位距离能耗（电量/距离）；
- v：行驶速度（距离/时间）；
- P_{charge}：充电功率（电量/时间）。

7.2 单段行驶：能耗与时间

对任意一段最短路距离 d，行驶时间与能耗分别为

$$t_{\text{travel}} = \frac{d}{v}, \qquad E(d) = \rho d. \tag{6}$$

若段起点电量为 B_{start}，且可直达，则段终点电量

$$B_{\text{end}} = B_{\text{start}} - \rho d, \qquad \rho d \le B_{\text{start}}. \tag{7}$$

7.3 直达判定与“先充后送”判定

设当前段为 $u \to w$，其最短路距离为 d_{uw}。

- **直达条件**：$\rho d_{uw} \le B_{\text{now}}$。
- **若不满足直达**：尝试经某充电站 c，并要求

$$\rho d_{uc} \le B_{\text{now}}, \qquad \rho d_{cw} \le B_{\max}. \tag{8}$$

第一式保证“当前电量可到站”，第二式保证“满电后可到目标”。

7.4 充电时长与充后电量

车辆到站时电量

$$B_{\text{arr}} = B_{\text{now}} - \rho d_{uc}. \tag{9}$$

充至满电所需能量缺口与时长

$$\Delta B = \max\{0,\ B_{\max} - B_{\text{arr}}\}, \qquad t_{\text{charge}} = \frac{\Delta B}{P_{\text{charge}}}. \tag{10}$$

充电结束后按实现取

$$B_{\text{after}} = B_{\max}. \tag{11}$$

7.5 排队与开始充电时刻

设到站时刻为 t_{arrive}。充电站并发槽位数为 2，开始充电时刻为

$$t_{\text{start}} = \min\{t \ge t_{\text{arrive}} :\ \text{该时刻占用会话数} < 2\}. \tag{12}$$

因此结束时刻

$$t_{\text{end}} = t_{\text{start}} + t_{\text{charge}}. \tag{13}$$

7.6 经站补能后的到达时刻与终点电量

若选择 $u \to c \to w$，则到达目标时刻

$$t_{\text{arrive},w} = t_0 + \frac{d_{uc}}{v} + (t_{\text{start}} - t_{\text{arrive}}) + t_{\text{charge}} + \frac{d_{cw}}{v}. \tag{14}$$

目标点电量

$$B_w = B_{\max} - \rho d_{cw}. \tag{15}$$

7.7 充电站选择准则（代码实现口径）

当直达不可行时，在所有可行充电站集合 $\mathcal{C}_{\text{feasible}}$ 中，选择使到达目标时刻最小者：

$$c^* = \arg\min_{c \in \mathcal{C}_{\text{feasible}}} \left(t_0 + \frac{d_{uc}}{v} + \text{queue_wait}(c, t_{\text{arrive}}) + \frac{\max(0,\ B_{\max} - B_{\text{arr}})}{P_{\text{charge}}} + \frac{d_{cw}}{v} \right). \tag{16}$$

即不仅看“最近充电站”，而是综合**到站路程、排队等待、充电时长、充后到目标路程**的最早到达准则。